

Plan de Desarrollo de Software

NexaSafe — Marco Scrum con modelo de entrega DevSecOps

NEXASAFE

PROGRAMA

Ingeniería de Sistemas

EQUIPO

Juan Felipe Pineda Cardona · Integrante 2 · Integrante 3

VERSIÓN DEL DOCUMENTO

1.0

INSTITUCIÓN

Fundación Universitaria San Mateo

FECHA DE ENTREGA

22 de septiembre de 2026

DOCUMENTO BASE

PLAN_NEXASAFE.md v1.1

1. Introducción y objetivo del plan

Este Plan de Desarrollo de Software define **cómo** el equipo construirá NexaSafe: el modelo de ciclo de vida elegido, la organización del equipo, las ceremonias, los artefactos, los criterios de entrada/salida de una historia de usuario y la forma en que la seguridad se integra como parte innegociable del proceso (DevSecOps), no como una fase posterior. Es el documento de referencia para planificar, ejecutar y verificar el trabajo de cada sprint entre el 22 de septiembre y el 28 de noviembre de 2026.

Scrum

MARCO ÁGIL

2 sem.

DURACIÓN DEL SPRINT

2-3

INTEGRANTES DEL EQUIPO

DevSecOps

MODELO DE ENTREGA

2. Modelo de ciclo de vida

2.1 Modelo seleccionado: Scrum

Se adopta **Scrum** como marco de trabajo ágil para el desarrollo de NexaSafe. La elección responde a tres factores del contexto del proyecto: (1) los requisitos evolucionan a medida que se valida cada incremento con el docente y con el "cliente simulado" del colegio; (2) el equipo es reducido (2–3 integrantes) y necesita ciclos cortos con retroalimentación frecuente para corregir el rumbo sin acumular riesgo; (3) la naturaleza sensible del dominio (datos de menores, alertas de seguridad física) exige que cada incremento sea auditable y demostrable, lo cual encaja con la cadencia de Sprint Review de Scrum.

2.2 Modelo de entrega: DevSecOps

Sobre el marco Scrum se superpone un modelo de entrega **DevSecOps** con principio rector *shift-left*: la seguridad no es una fase previa a la entrega, es un criterio de la Definition of Done que se ejecuta automáticamente en cada commit (ver capítulo 7). Este modelo obliga a que, desde el Sprint 0, exista pipeline de integración continua con controles de seguridad automatizados, antes de construir la primera historia de negocio.

2.3 Por qué no cascada ni Kanban puro

Modelo	Razón de descarte
Cascada	Exige requisitos cerrados desde el inicio; el proyecto depende de decisiones que solo se validan con el uso (p. ej. tolerancia del corredor seguro, umbral de sacudida para el pánico).
Kanban puro	No exige una revisión periódica formal con el Product Owner; en un entregable académico con fechas de sustentación fijas, el ritmo de Sprint Review/Retrospective da puntos de control necesarios.

3. Organización del equipo

3.1 Roles

Rol	Responsable	Responsabilidades
Product Owner	Docente del proyecto integrador (o representante del colegio como cliente simulado)	Prioriza el backlog, acepta o rechaza el incremento en la review, resuelve dudas de negocio.
Scrum Master	Integrante rotativo del equipo	Facilita ceremonias, retira impedimentos, protege el sprint de cambios de alcance.
Development Team	3 integrantes: Móvil, Backend/Datos, DevSecOps/QA	Equipo multifuncional y autoorganizado. Cada integrante rota por el rol de <i>Security Champion</i> cada dos sprints.

Ajuste por tamaño de equipo

El equipo es de 2 a 3 integrantes. Con 3, se mantiene la velocidad objetivo de 24 puntos por sprint. Con 2, se ajusta a **16–18 puntos por sprint**, se combinan los roles de Dev y Scrum Master, y las historias **SHOULD** se sacrifican primero.

3.2 Convención de trabajo

Cada historia de usuario es un issue en GitHub con plantilla que incluye título en formato de historia, criterios de aceptación en Gherkin, estimación en puntos y etiquetas (`epic:*`, `type:*`, `security`). La propiedad del código es colectiva: no hay conocimiento aislado en una sola persona (mitigación del riesgo R7, capítulo 8).

4. Ceremonias y cadencia

4.1 Ciclo del sprint

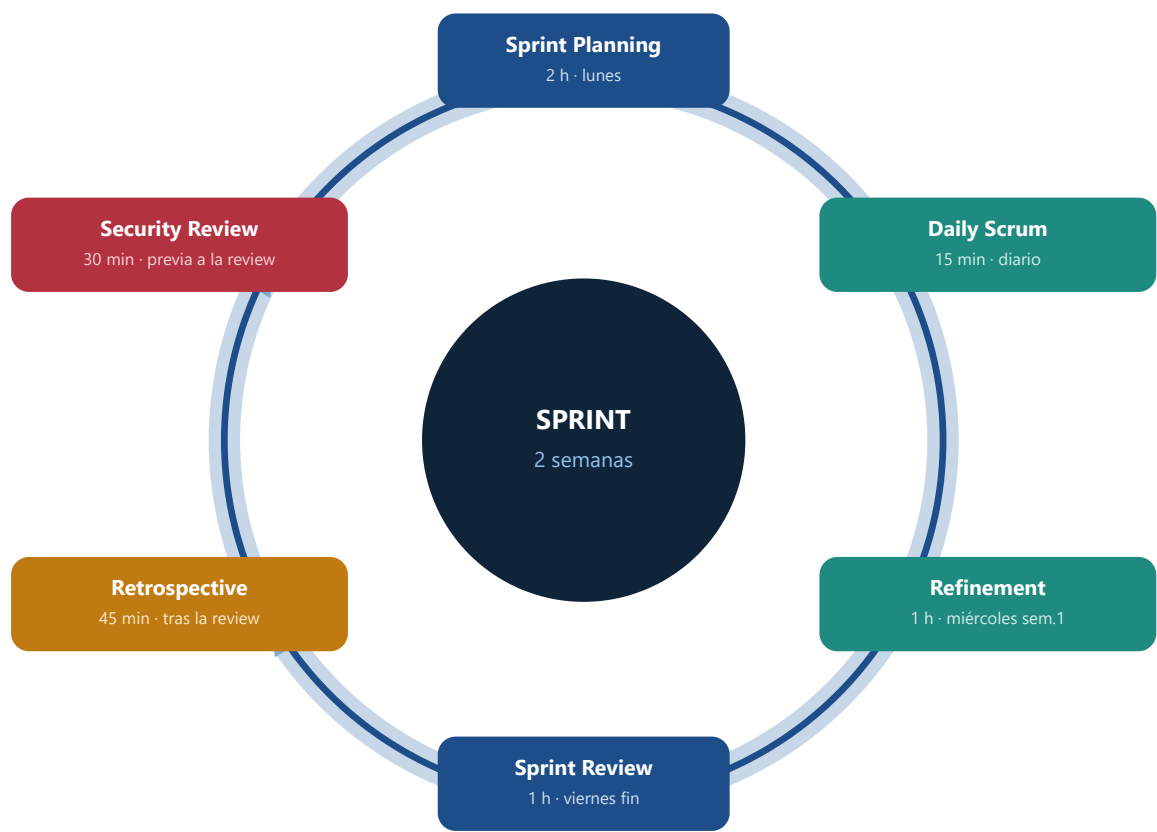


Figura 4.1 — Ciclo de ceremonias dentro de un sprint de dos semanas.

4.2 Detalle de ceremonias

Ceremonia	Duración	Frecuencia	Salida
Sprint Planning	2 h	Inicio de sprint (lunes)	Sprint Goal + Sprint Backlog comprometido
Daily Scrum	15 min	Diario (asíncrono por Discord si no hay presencialidad)	Impedimentos identificados
Refinement	1 h	Miércoles de la semana 1	Historias del siguiente sprint estimadas y con DoR
Sprint Review	1 h	Fin de sprint (viernes)	Incremento demostrado y feedback del PO
Sprint Retrospective	45 min	Fin de sprint, después de la review	1–2 acciones de mejora con responsable
Security Review	30 min	Fin de sprint, previa a la review	Hallazgos del pipeline triados y priorizados en backlog

4.3 Cadencia

- **Duración del sprint:** 2 semanas.
- **Sprint 0** dedicado a habilitación técnica y de seguridad (no produce valor de negocio, produce plataforma).
- **Sprint Goal único por sprint**, redactado en una sola frase orientada a resultado.

5. Artefactos Scrum

Artefacto	Herramienta	Compromiso asociado
Product Backlog	GitHub Projects (vista <i>Backlog</i>)	Product Goal
Sprint Backlog	GitHub Projects (vista <i>Sprint actual</i> : To Do / In Progress / In Review / Done)	Sprint Goal
Incremento	Build firmado en EAS + proyecto Supabase de staging migrado	Definition of Done
Impediment Log	Issues con etiqueta <code>impediment</code>	—
Risk & Security Register	<code>/docs/security/risk-register.md</code> versionado en el repo	—

6. Definition of Ready y Definition of Done

6.1 Definition of Ready — para entrar al sprint

- Historia redactada en formato *Como... quiero... para...*
- Criterios de aceptación en Gherkin (Dado/Cuando/Entonces)
- Estimada por el equipo en puntos (Fibonacci)
- Dependencias técnicas identificadas y desbloqueadas
- Impacto en datos personales evaluado (¿toca datos de menores? ¿ubicación?)
- Diseño/wireframe disponible si la historia tiene UI
- Cabe holgadamente en un sprint (≤ 8 puntos; si no, se divide)

6.2 Definition of Done — para cerrar una historia

Funcional

- Todos los criterios de aceptación verificados
- Probada en dispositivo Android físico, no solo en emulador

Calidad de código

- Pull Request revisado y aprobado por al menos un par
- Cobertura de pruebas unitarias $\geq 70\%$ en el módulo tocado
- Lint y formateo sin errores (ESLint + Prettier)
- Sin TODO ni código comentado en la rama principal

Seguridad — obligatorio, no negociable

- SAST (Semgrep) sin hallazgos *High* o *Critical*
- SCA (npm audit) sin vulnerabilidades *Critical* explotables
- Escaneo de secretos (Gitleaks) limpio
- Ningún dato personal en logs (verificado manualmente en el PR)
- Tablas y Edge Functions nuevas con RLS, autorización por rol y rate limiting

Documentación y entrega

- README / CHANGELOG actualizados
- Tipos de Supabase regenerados y políticas RLS actualizadas si cambió el esquema
- Desplegado en el ambiente de staging por el pipeline
- Demostrable en la Sprint Review

7. Estrategia DevSecOps — resumen ejecutivo

El detalle completo del pipeline, el modelado de amenazas y los controles específicos del producto vive en el Documento de Diseño Arquitectónico y en `PLAN_NEXASAFE.md` (secciones 9–11). Este capítulo resume los elementos que afectan directamente la planificación del sprint.

7.1 Controles por fase del ciclo

Fase	Práctica	Herramienta	Cuándo
Plan	Modelado de amenazas STRIDE por épica	OWASP Threat Dragon	Refinement
Code	Pre-commit hooks, revisión de pares obligatoria	pre-commit, Gitleaks	Local / antes del merge
Build	SAST, SCA, SBOM	Semgrep, CodeQL, npm audit, Syft	En cada PR / cada release
Test	Unitarias, integración, E2E, DAST, MobSF	Jest, RNTL, Maestro, OWASP ZAP, MobSF	Cada PR / nocturno / pre-release
Release	Firma del artefacto, gestión de secretos	EAS Build + keystore, GitHub/EAS Secrets	Al taggear versión
Deploy	Verificación de políticas RLS antes de aplicar	Supabase CLI + suite de pruebas de política	Antes del despliegue
Operate / Monitor	Logs sin PII, auditoría append-only, errores	JSON logging, tabla <code>audit_log</code> , Sentry	Runtime continuo

7.2 Política de tratamiento de hallazgos

Severidad	SLA de corrección	Efecto
CRITICAL	24 h	Bloquea el merge y el despliegue
HIGH	Dentro del sprint actual	Bloquea el merge
MEDIUM	Siguiente sprint	Advertencia; entra al backlog como issue security
LOW	Backlog priorizado	Registrado, sin bloqueo

Toda excepción se documenta en `/docs/security/exceptions.md` con justificación, responsable y fecha de vencimiento. **No hay excepciones permanentes.**

7.3 Rol de Security Champion

Un integrante del Development Team asume el rol por dos sprints: facilita el modelado de amenazas en el refinement, tría los hallazgos del pipeline antes de la Security Review, mantiene el registro de riesgos y las excepciones, y presenta el estado de seguridad en la Sprint Review (3 minutos fijos).

8. Gestión de riesgos del proyecto

ID	Riesgo	Prob.	Impacto	Respuesta
R1	La ubicación en segundo plano se detiene por optimización de batería del fabricante (Xiaomi, Huawei).	Alta	Alto	Investigación temprana en Sprint 2; guía de exclusión de optimización; foreground service persistente.
R2	Falsas alarmas frecuentes erosionan la confianza y la gente deja de atender.	Media	Alto	Corredor de tolerancia configurable, temporizador de confirmación, sin sanción por falsa alarma.
R3	El colegio no designa responsable del puesto de control.	Media	Alto	Diseñar el flujo para que funcione solo con la red de apoyo si no hay institución activa.
R4	Fuga de datos de menores.	Baja	Crítico	Cifrado, minimización, retención corta, RBAC, auditoría, pentest interno en Sprint 6.
R5	El número de prueba de WhatsApp Cloud API solo alcanza 5 destinatarios verificados.	Alta	Medio	Restricción de diseño (máx. 5 contactos de seguridad); el push permanece como canal principal sin ese límite.
R6	Alcance excesivo para el tiempo disponible del semestre.	Alta	Alto	MoSCoW estricto; las historias COULD se sacrifican primero; revisión de alcance en el hito H3.
R7	Rotación o baja disponibilidad de un integrante.	Media	Medio	Propiedad colectiva del código, documentación en el repo, sin conocimiento aislado en una persona.
R8	Falsos positivos del disparo por sacudida (correr, bus, guardar el celular).	Alta	Medio	Umbral calibrado + ventana de cancelación por PIN; se activa solo en modo trayecto; degradar a COULD si no es viable.

9. Métricas de seguimiento

9.1 Métricas Scrum

Métrica	Objetivo	Cómo se mide
Velocidad	Estable ± 15 % entre sprints	Puntos completados por sprint
Cumplimiento del Sprint Goal	≥ 85 % de los sprints	Aceptación en la review
Trabajo no planificado	≤ 15 % del sprint	Puntos de issues agregados en curso
Burndown	Descendente sin mesetas prolongadas	GitHub Projects

9.2 Métricas DevOps (DORA)

Métrica	Objetivo
Frecuencia de despliegue a staging	≥ 1 por día hábil
Lead time (commit $\rightarrow$ staging)	< 30 min
Tasa de fallo en cambios	< 15 %
Tiempo de recuperación (MTTR)	< 2 h

9.3 Métricas de seguridad

Métrica	Objetivo
Vulnerabilidades Critical/High abiertas al cierre del sprint	0
Tiempo medio de remediación de hallazgos High	< 5 días
Cobertura del pipeline de seguridad	100 % de los PR
Secretos filtrados detectados en main	0

10. Plan de comunicación

Canal	Propósito	Frecuencia
Discord del equipo	Daily Scrum asíncrono, coordinación rápida	Diaria
GitHub Projects	Estado del Sprint Backlog y del Product Backlog	Continua
GitHub Issues / PR	Discusión técnica trazable de cada historia	Continua
Reunión de Sprint Review	Demostración del incremento y feedback del PO	Cada 2 semanas
Acta de retrospectiva	Registro de acciones de mejora con responsable	Cada 2 semanas